REFERENCE REPOSITORY

GitHub: https://github.com/SETAPESU26/L5_19_music_catalog

Submission: https://github.com/satk4ll/PES1UG24AM257_SE (Lab 5 folder)

LAB 5: STATIC CODE ANALYSIS

Detailed Lab Report — Music Catalog Python Program

Student	Satyanarayana Kallakuri
Lab	Lab 5 – Static Code Analysis
Program	Music Catalog Python Program

1. Introduction

Static code analysis examines source code without executing the program. It helps developers identify potential programming mistakes, security risks, and style inconsistencies early in development. Finding issues before delivery can make software easier to understand, safer to maintain, and less likely to fail in common situations. This lab applies Pylint, Bandit, and Flake8 to a Python music-catalog program. The program uses SQLite to store song information and includes functions for adding songs, selecting a song, classifying popularity, serializing a playlist, and calculating total plays.

2. Objectives

- Understand static analysis and how it differs from testing by execution.
- Use Pylint to review code quality, naming, and common programming practices.
- Use Bandit to identify potentially unsafe security-related coding patterns.
- Use Flake8 to check Python formatting and style conventions.
- Correct issues while preserving the intended catalog functionality.
- Compare before-and-after reports to assess the changes.

3. Tools Used

Tool	Role	Typical findings
Pylint	Reviews code quality, naming, common mistakes, and maintainability.	Mutable defaults, broad exceptions, naming conventions, and code organization.
Bandit	Scans Python source for common security risks and unsafe coding patterns.	Risky APIs or patterns, including unsafe deserialization and unsuitable random-number use in security contexts.
Flake8	Checks Python style and formatting, including whitespace and file-ending conventions.	Whitespace problems, formatting violations, and blank lines at the end of a file.

4. Program Overview

The program creates an in-memory SQLite database with a songs table containing a song ID, title, artist, and play count. Its functions add songs to the database and playlist, choose a random song ID, classify popularity by play count, save and load playlists, and calculate total plays. The revised version uses snake_case function names, JSON for playlist data, parameterized SQL, and explicit handling of an empty catalog.

Function	Purpose
`add_song()`	Inserts a song into SQLite and appends its title to a playlist.
`shuffle_pick()`	Selects a random song ID, or returns None when the catalog is empty.
`popularity()`	Returns popular, niche, or unknown based on stored play count.
`save_playlist()` / `load_playlist()`	Serializes and restores playlist data using JSON.
`total_plays()`	Returns the sum of play counts, handling an empty result and SQLite errors.

5. Methodology

Source inspection: Reviewed the assigned script to understand the SQLite table, playlist behavior, and function flow.

Environment setup: Created and activated a Python virtual environment and installed Pylint, Bandit, and Flake8.

Baseline analysis: Ran each tool on the original source and saved the output as before-reports.

Code corrections: Improved SQL parameter handling, playlist defaults, serialization, exception handling, naming, and edge-case behavior.

Post-change analysis: Generated after-reports. The shared terminal output showed the improved Pylint score and Bandit result; Flake8's final clean result should be verified after the last edit.

6. Issues Identified and Corrections

Issue	Original concern	Correction	Benefit
Mutable default	A default such as `playlist=[]` may be shared across calls.	Use None and initialize a new list inside the function.	Prevents unintended sharing of playlist state.
SQL query construction	String-formatted SQL can mix values into query syntax.	Use placeholders and pass values separately.	Reduces SQL-injection risk and clarifies query handling.
Pickle deserialization	Loading untrusted pickle data can be dangerous.	Use JSON for a playlist containing strings and validate loaded data.	Avoids unsafe pickle object reconstruction for this use case.
Broad exception	Bare except can hide unrelated failures.	Catch the expected SQLite exception type.	Makes failures easier to diagnose.
Empty catalog	Random selection fails if no IDs exist.	Check for an empty ID list and return None.	Handles an edge case without an exception.
Naming conventions	CamelCase function names differ from common Python style.	Use snake_case names.	Improves readability and consistency.
File formatting	Flake8 reported W391 for a blank line at end of file during an earlier run.	Edited the file ending and regenerated reports; run Flake8 again for final confirmation.	Keeps source formatting consistent.

7. Verification Results

Tool	Observed result	Interpretation
Pylint	Latest shared terminal output: **10.00/10**; earlier score shown was 4.57/10 and then 9.78/10.	The score exceeds the lab target of 9.5/10.
Bandit	Latest shared output: **No issues identified**.	No findings were reported in that scan.
Flake8	W391 appeared in an earlier check. The final clean output was not clearly visible in the last screenshot.	Run `python -m flake8 music_catalog.py`; no output indicates no violations.

8. Discussion

The three tools provide complementary checks. Code can be neatly formatted but still use a risky API, or it can avoid obvious security findings while remaining difficult to maintain. Using multiple tools therefore gives a broader review than relying on one checker. Findings should be interpreted in context and fixed meaningfully rather than hidden simply to improve a score.

Parameterized queries separate data from SQL syntax. JSON is suitable for this example because the playlist contains simple strings, though application-level validation is still important. Specific exception handling makes expected database errors manageable without masking unrelated programming mistakes. Explicitly checking for an empty catalog also makes random selection more predictable.

9. Conclusion

This lab applied Pylint, Bandit, and Flake8 to a Python music-catalog program and used the analysis to improve code quality, security practices, and consistency. The latest shared outputs showed a Pylint score of 10.00/10 and no Bandit issues. Flake8 should be run once more after the final edit to confirm that the earlier W391 warning is resolved. Overall, static analysis supports clearer, safer, and more maintainable code.